

Fundamentals of System Security (COMSM0122)

Workshop 2: Infrastructure Evaluation based on RICE Principles

Prerequisite:

None

Overview

Infrastructure as Code (IaC) enables consistent, automated provisioning and management of cloud resources. To ensure long-term maintainability and operational excellence, IaC should adhere to the RICE principles: Reproducibility, Idempotency, Composability, and Evolvability. These principles promote reliable deployments across environments, safe reapplication of configurations, modular architecture design, and adaptability to changing requirements. By embedding RICE into infrastructure workflows, development teams can build systems that are robust, scalable, and aligned with modern DevOps practices.

In this workshop, you will learn to evaluate a system infrastructure description against the RICE principles. This is based on a case study that involves the design of an AWS-based web Application Architecture. The case study involves a natural-language description of 5 design scenarios. You will analyse each scenario to identify which principle has been satisfied. At the end of your scenario analysis, you will generate a ranking based on the maturity level of the scenario and RICE satisfaction.

Note. The scenarios are sometimes intentionally described using AWS domain-specific concepts. This is necessary to reduce the ambiguity of the scenarios described. You can look up concepts that you are not familiar with using AWS glossary and documentation using the following links:

- <https://docs.aws.amazon.com/glossary/latest/reference/glos-chap.html>
- <https://docs.aws.amazon.com>

Scenario 1:

In this scenario, the AWS-based web application architecture is designed to satisfy the following properties:

Networking Architecture

- The system uses the default AWS VPC, without custom subnetting or routing.
- No segmentation between public and private subnets is implemented.
- An EC2 instance is directly exposed to the internet via a public IP address.
- No NAT Gateway or Internet Gateway configuration is explicitly defined.
- Security Groups are created manually, with permissive rules and no documentation.

Compute Resources

- A single Amazon EC2 instance is provisioned manually through the AWS Management Console.
- The instance hosts a custom Apache web server, configured via SSH without automation.
- No Auto Scaling or Load Balancer is used, limiting availability and scalability.

Storage and Database Services

- Static assets and application data are stored on the EC2 instance's local disk.

- A MySQL database is installed directly on the same EC2 instance.
- No backup strategy or managed database service (e.g., RDS) is implemented.

Security and Access Control

- IAM roles and policies are not used; access is managed manually.
- SSH access is configured without key rotation or centralised secrets management.
- Security groups allow broad access and are not tagged or version-controlled.

Monitoring and Logging

- No centralised logging or monitoring tools (e.g., CloudWatch) are configured.
- System health is checked manually by logging into the EC2 instance.
- No alerting or automated diagnostics are in place.

Tagging and Environment Management

- Resources are not tagged, making cost tracking and resource ownership unclear.
- No separation between development, staging, and production environments.
- All changes are made manually, with no version control or change history.

Scenario 2:

In this scenario, the AWS-based web application architecture is designed to satisfy the following properties:

Networking Architecture:

- A Virtual Private Cloud (VPC) is provisioned across two Availability Zones to ensure fault tolerance and redundancy.
- The VPC includes:
 - Public subnets for hosting internet-facing components such as load balancers.
 - Private subnets for backend services, including application servers and databases.
- An Internet Gateway (IGW) is attached to the VPC to enable outbound internet access from public subnets.
- A NAT Gateway is deployed in a public subnet to allow private instances to initiate outbound traffic securely.
- Route tables are configured to manage traffic flow between subnets and external endpoints.
- DNS support is enabled to facilitate internal name resolution.

Compute Resources

- Amazon EC2 instances are deployed in private subnets to host the application backend.
- An Auto Scaling Group (ASG) is configured to dynamically adjust the number of EC2 instances based on traffic load and performance metrics.
- A Load Balancer is deployed in the public subnet to distribute incoming traffic across EC2 instances.

Storage and Database Services

- Amazon S3 is used for storing static assets, backups, and logs.
- A Relational Database Service (RDS) instance running MySQL is provisioned in private subnets with multi-AZ support for high availability.
- A DB Subnet Group ensures the database spans multiple Availability Zones.

Security and Access Control

- Security Groups are defined to control inbound and outbound traffic at the instance level.

- IAM roles and policies are used to manage access to AWS services and enforce least privilege.
- Network ACLs provide an additional layer of subnet-level traffic filtering.

Monitoring and Logging

- VPC Flow Logs are enabled to capture network traffic for auditing and troubleshooting.
- Amazon CloudWatch is integrated to monitor system metrics, set alarms, and visualise performance trends.
- CloudWatch Logs stores application and infrastructure logs for centralised analysis.

Tagging and Environment Management

- All resources are tagged with standardised keys such as **Environment**, **Owner**, and **Project** to support cost allocation, automation, and governance.
- The architecture supports multiple environments (e.g., development, staging, production) through parameterised configurations and modular design.

Scenario 3:

In this scenario, the AWS-based web application architecture is designed to satisfy the following properties:

Networking Architecture:

- The application is hosted using the default AWS VPC, without custom subnetting or multi-AZ design.
- No dedicated public or private subnets are defined; resources rely on default networking behaviour.
- Security Groups are manually created to allow HTTP/HTTPS access to the S3 bucket and CloudFront distribution.
- No NAT Gateway or custom route tables are configured.

Compute Resources

- No EC2 instances or container services are used.
- The application is purely static and served via Amazon S3 and CloudFront, eliminating the need for backend compute.

Storage and Content Delivery

- Amazon S3 hosts static website files (HTML, CSS, JS) with public read access enabled via bucket policy.
- Amazon CloudFront is manually configured to cache and distribute content globally.
- AWS Certificate Manager (ACM) provides SSL certificates for HTTPS access.
- Amazon Route 53 manages DNS records for the application domain.

Security and Access Control

- IAM roles and policies are manually created to manage access to S3 and CloudFront.
- Bucket policies are manually edited to allow public access.
- No centralised secrets management or automated policy enforcement is in place.

Monitoring and Logging

- Basic logging is enabled for CloudFront and S3 access.
- No integration with Amazon CloudWatch or third-party monitoring tools.
- Logs are stored in S3 but not actively analysed or visualised.

Tagging and Environment Management

- Resources are tagged manually, if at all, with inconsistent naming conventions.
- No environment separation (e.g., dev/staging/prod) is implemented.
- All configuration is performed manually via the AWS Console or CLI.

Scenario 4

In this scenario, the AWS-based web application architecture is designed to satisfy the following properties:

Networking Architecture

- A **default AWS VPC** is used without custom CIDR blocks, subnet segmentation, or multi-AZ design.
- All resources are deployed in a single Availability Zone, increasing the risk of downtime.
- **Security Groups** are manually created and modified without version control or naming conventions.
- No NAT Gateway or custom route tables are configured; internet access is granted via public IPs.

Compute Resources

- A single **Amazon EC2 instance** is manually launched via the AWS Console.
- The instance hosts both the web application and database, violating separation of concerns.
- Configuration is performed imperatively via SSH, including package installation and environment setup.
- No Auto Scaling or Load Balancer is used, limiting scalability and fault tolerance.

Storage and Database Services

- The application stores data locally on the EC2 instance's root volume.
- A MySQL database is installed directly on the same EC2 instance.
- No backups, snapshots, or managed database services (e.g., RDS) are configured.

Security and Access Control

- IAM roles and policies are not used; access is managed manually through the AWS Console.
- SSH access is configured with static key pairs and no rotation policy.
- Security groups allow broad access (e.g., open ports to 0.0.0.0/0) and are not tagged or documented.

Monitoring and Logging

- No centralised logging or monitoring tools (e.g., CloudWatch) are integrated.
- Application logs are stored locally and accessed manually via SSH.
- No alerting or performance metrics are collected.

Tagging and Environment Management

- Resources are not consistently tagged, making cost tracking and ownership unclear.
- No separation between development, staging, and production environments.
- All changes are made manually, with no automation or Infrastructure as Code (IaC) tooling.

Scenario 5

In this scenario, the AWS-based web application architecture is designed to satisfy the following properties:

Networking Architecture

- A single AWS VPC is created manually with one public subnet and no private subnet segmentation.
- All resources—including web servers and databases—are placed in the same public subnet.
- Security Groups are manually configured with overly permissive rules (e.g., open ports to all IPs).
- No NAT Gateway or route table customisation is implemented.
- DNS resolution is handled manually via public IPs, without Route 53 integration.

Compute Resources

- Two Amazon EC2 instances are manually launched to host the frontend and backend services.
- Each instance is configured via SSH with hard-coded IP addresses and environment variables.
- No Auto Scaling Group or Load Balancer is used.
- Application logic assumes static IPs and direct hostnames, creating tight coupling between services.

Storage and Database Services

- A MySQL database is manually installed on one of the EC2 instances.
- Application code directly references the database using its IP address and hard-coded credentials.
- No RDS or managed storage services are used.
- No backup or snapshot strategy is in place.

Security and Access Control

- IAM roles and policies are not used; access is managed manually through the AWS Console.
- SSH keys are reused across all instances without rotation or access control.
- Security groups lack descriptive names and tagging, making auditing difficult.

Monitoring and Logging

- No centralised logging or monitoring tools (e.g., CloudWatch) are configured.
- Logs are stored locally on EC2 instances and accessed manually.
- No alerting or performance metrics are collected.

Tagging and Environment Management

- Resources are inconsistently tagged, if at all.
- No separation between development, staging, and production environments.
- All configuration is performed manually, with no version control or automation.

Tasks

- 1) For each scenario:
 - a) Generate an infrastructure diagram that describes the scenario.
 - b) Carry out an assessment of each scenario against the RICE Principles – describing how the scenario satisfies or does not satisfy each of the principles by completing the table below. The evaluation column should demonstrate the extent the scenario is aligned (or not aligned) with RICE.

Principle	Satisfied (Yes, No, Partially)	Evaluation
Reproducibility		
Idempotency		
Composability		
Evolvability		

- c) Write a short (max 100 words) summary of the scenario, describing its level of maturity and alignment with modern cloud architecture principles, suitability for iterative development, operational scalability, and enterprise-grade deployment.
- 2) Using the keys below, generate an infrastructure maturity and RICE satisfaction ranking based on all 5 scenarios by completing the table below.

Rank	Scenario	Reproducibility	Idempotency	Composability	Evolvability	Maturity Level

Key

Rank: 1 | 2 | 3 | 4 | 5

Scenario: 1 | 2 | 3 | 4 | 5

Reproducibility/Idempotency/Composability/Evolvability: Yes | No | Partially)

Maturity Level:

Enterprise-grade / Production-ready |

Basic / Prototype-ready |

Ad hoc / Risk-prone |

Unstable / Poorly modularised |

Legacy / Fragile

Deliverables

This is a formative assessment. Ensure one of the TAs or the course lecturer has reviewed your solutions before the end of the lab session.